

Организация бесшовной миграции на 64-битные ключи

В базах данных, используемых для обслуживания потребностей сервисов на платформе Wasaby, поддерживается использование целочисленных полей (в том числе в составе первичных и внешних ключей) разной размерности (32 и 64 бита). Предполагается, что в зависимости от степени наполненности и характера домена данных конкретной таблицы, ответственный разработчик выберет размерность для обеспечения наиболее эффективного использования дискового пространства и производительности запросов. Однако в ходе эксплуатации и эволюции программных решений может возникать ситуация, когда изначальная оценка оказывается неверной.

В типичном случае возникает потребность в "расширении" поля, т.е. выполнении перехода от 32-битного целочисленного поля (как правило - первичного ключа) к 64-битному. Несмотря на то, что в PostgreSQL предусмотрена штатная возможность изменения типа поля, она, во-первых, сопряжена с (очень-очень) длительной access-exclusive блокировкой таблицы, при которой пользователи вообще никак не могут работать с ее данными, а во-вторых, зачастую (обычно на больших таблицах, на которых данная процедура как правило и применяется) упирается в ограничение на число блокировок на транзакцию и завершается ошибкой. Т.к. длительный (а речь тут идет о днях) простой продуктивных баз, к тому же не гарантирующий успеха, является неприемлемым с т.з. потребительских характеристик продуктов компании, приходится выполнять данную операцию разного рода обходными способами.

Общая схема процесса выглядит примерно так:

1. Для каждого "расширяемого" поля в таблице создается временное "вспомогательное" поле нужной размерности. Изначально во всех записях таблицы это поле не заполнено.
2. На каждую таблицу, в которой были созданы "вспомогательные" поля, навешивается триггер, обеспечивающий синхронизацию значений "основного" и "вспомогательного" полей при внесении изменений (в т.ч. вставке новых записей) в данные.
3. Постепенно маленькими пачками выполняется заполнение "вспомогательных" полей значениями "расширяемых" для всех ранее существовавших ранее записей.
4. В concurrently-режиме создаются дублирующие индексы, включающие "вспомогательные" поля вместо "расширяемых", а также индексы, соответствующие первичным ключам и ограничениям типа unique.
5. Производится переименование "вспомогательных" полей в "расширяемые", а старые "расширяемые" поля удаляются (а вместе с ними каскадно удаляются старые индексы и ограничения целостности).

Подробнее про схему миграции можно почитать [здесь](#). Если все подготовительные мероприятия были выполнены верно, то шаг 5 вышеприведенного алгоритма выполняется достаточно быстро (за несколько десятков секунд) и, соответственно, обеспечивается минимальный простой сервиса.

Однако, как легко заметить, данный алгоритм является достаточно сложным, особенно в случае, если требуется обеспечить устойчивость скрипта миграции к ошибкам и возможность его прерывания и возобновления с сохранением достигнутого прогресса. При этом в последнее время миграции подобного рода стали довольно распространенным явлением, поэтому для упрощения процедуры был разработан вспомогательный платформенный каркас, который состоит из двух частей:

1. Специализированного метода БЛ, подготавливающего необходимую переходную инфраструктуру без простоя сервисов, использующих БД.
2. Ответной части в ядре конвертера БД, которая "распознает" созданные вспомогательные объекты и использует их для быстрого выполнения миграции в точке конвертации БД.

Далее рассмотрим каждую часть чуть более подробно.

Метод БЛ

"DBConverter.PreparePKMigrationInCurrentSchema"

Первым компонентом вышеописанного каркаса является метод БЛ `DBConverter.PreparePKMigrationInCurrentSchema`. Именно с его помощью осуществляется подготовка к миграции.

Метод размещается в модуле БЛ "DB Converter Postgres" (входит в состав SDK платформы), соответственно, чтобы им воспользоваться, в проект вашего сервиса необходимо подключить данный модуль и все его зависимости. Возможно, что это потребует от вас выпуска ХФ сервиса. Метод является служебным (т.е. его нельзя позвать "снаружи" сервиса). Предполагается, что применяться он будет только в скриптах "Хоттабыча".

В качестве единственного параметра метод принимает объект типа Record. В составе объекта должны (или могут) быть следующие поля:

1. **ColumnsToMigrate** - обязательное поле, список записей. В данном поле перечисляется список колонок таблиц, которые необходимо "расширить". Вложенный RecordSet должен состоять из двух строковых полей: **Table** (таблица, которой принадлежит колонка) и **Column** (собственно имя "расширяемой" колонки). Список ColumnsToMigrate не обязан быть "исчерпывающим". Все поля всех таблиц, которые связаны реляционными отношениями с полями, перечисленными в ColumnsToMigrate, также автоматически будут взяты в работу.
2. **BatchSize** - необязательное поле, целое число. Задаёт размер пачки записей, обрабатываемых (путем синхронизации "переходных" полей с исходными) за один запрос к одной таблице. Если не будет передано, то будет применяться значение по умолчанию - 10000. Оптимальное значение данного параметра надо подбирать эмпирическим путем, чтобы выполняемые UPDATE-ы не порождали много блокировок, мешающих "основной" деятельности БД. С другой стороны, излишнее уменьшение данного значения ведет к увеличению времени исполнения скрипта.
3. **NumThreads** - необязательное поле, целое число. Задаёт число параллельно обрабатываемых (или создаваемых) вспомогательных объектов. Под объектами здесь понимаются таблицы и индексы. По умолчанию имеет значение 1. "Распараллеливание" в NumThreads производится на фазе синхронизации значений основных и переходных полей, а также на фазе создания дублирующих индексов в concurrently режиме. Т.е. если, например, значение NumThreads = 2, то на фазе синхронизации метод будет параллельно делать UPDATE-ы записей в двух (разных) таблицах, а на фазе создания индексов - будет параллельно строить два индекса (но на разных таблицах, создание индексов на одной таблице не распараллеливается ввиду повышения вероятности конфликтов).
4. **ThreadsPerTable** - необязательное поле, список записей. Позволяет распараллелить обработку записей на фазе синхронизации значений основных и переходных полей для очень больших таблиц. Должен иметь два поля: строковое **Table** - название таблицы и целочисленное **NumThreads** - число потоков обработки для данной таблицы (имеет смысл только значения > 1). Если для таблицы T передана инструкция по распараллеливанию обработки в рамках значения данного параметра, то на фазе синхронизации значений основных и переходных полей для нее одновременно будет исполняться несколько update-запросов (на "BatchSize" каждый).
5. **BaseColumnsOnly** - необязательное поле, логическое значение. Позволяет отказаться от "достройки" списка из ColumnsToMigrate по реляционным связям. Использовать на свой страх и риск.
6. **ColumnsToSkip** - необязательное поле, список записей (того же формата, что и "ColumnsToMigrate"), позволяющий в ходе "достройки" списка из ColumnsToMigrate по реляционным связям "отсечь" часть веток. Использовать на свой страх и риск.

В целом, код метода реализует вышеописанный алгоритм из уже упоминавшейся выше [статьи](#). Схема работы следующая:

1. Сначала метод создает все вспомогательные поля и синхронизирующие триггеры;
2. Затем в **NumThreads** потоков исполняет синхронизацию данных в таблицах (также учитывая указания из "ThreadsPerTable");
3. Затем в **NumThreads** потоков создает дублирующие индексы (включая индексы для уникальных ограничений и ограничений типа "первичный ключ").

Весь достигнутый прогресс метод сохраняет в специализированной таблице public."DBObjectsTransitionState" (она же впоследствии используется ядром конвертера БД), что позволяет останавливать/перезапускать метод (и ВНР-ы на его основе) без потерь.

Пример вызова метода (из ВНР) для выполнения миграции таблиц ЭДО в 6 потоков:

```
1 param_format = sbis.RecordFormat()
2 param_format.AddRecordSet('ColumnsToMigrate')
3 param_format.AddInt32('NumThreads')
4 arg = sbis.Record(param_format)
5
6 rf = sbis.RecordFormat()
7 rf.AddString('Table')
8 rf.AddString('Column')
9 ColumnsToMigrate = sbis.RecordSet(rf)
10
11 to_migrate = {
12     'Событие' : ['@Событие'],
13     'Документ' : ['@Документ'],
14     'ВнешнийДокумент' : ['@ВнешнийДокумент'],
15     'ВерсияВнешнегоДокумента' : ['@ВерсияВнешнегоДокумента'],
16     'ВнешнийДокументСобытия' : ['@ВнешнийДокументСобытия'],
```

```

17         'ЭЦП': ['@ЭЦП'],
18         'СобытиеРасширение': ['@Событие'],
19         'ДокументРасширение': ['@Документ'],
20         'GUIDДокумента': ['@GUIDДокумента'],
21         'ЛицоДокумента': ['@ЛицоДокумента', 'Событие'],
22         'ДебетКредит': ['Документ'],
23         'СвязьДокументов': ['@СвязьДокументов'],
24     }
25
26     for table, fields in to_migrate.items():
27         for field in fields:
28             ColumnsToMigrate.AddRow(sbis.Record({'Table': table, 'Column':
29 field}))
30
31     arg.Set('ColumnsToMigrate', ColumnsToMigrate)
32     arg.Set('NumThreads', 6)
33
34     obj = sbis.BLObject('DBConverter')
35     obj.Invoke('PreparePKMigrationInCurrentSchema', arg)

```

Работа ядра конвертера БД по метаданным подготовленной миграции

В рамках реализации каркаса миграции было доработано ядро конвертера БД. Оно использует данные из таблицы `public.DBObjectsTransitionState` для оптимизации процедуры конвертации. В частности:

1. Автоматически созданные скриптом "переходные" поля делаются "заменой" расширяемым "старым" полям (т.е. не надо руками делать декларативное расширение конвертера).
2. Автоматически подхватываются все созданные скриптом дублирующие индексы.
3. Автоматически выполняется создание новых ограничений типа PK или Unique с использованием дублирующих индексов и конструкции `USING INDEX`.
4. Используется ускоренное создание новых внешних ключей в режиме `NOT VALIS`.

Записи об "использованных" в ходе конвертации объектах удаляются из `DBObjectsTransitionState`. Если же в данной конвертации какое-то из подготовленных преобразований не потребовалось (например, к релизу команда не успела подготовиться к "расширению" ключей и не внесла соответствующие правки в `disx`-файлы или наоборот, в старой версии нашелся критичный баг, требующий конвертации), то соответствующие вспомогательные объекты БД не будут удалены.

Проблемы при использовании каркаса

В ходе внедрения вышеописанного каркаса был выявлен ряд типовых проблем. О них поговорим в этом разделе.

Запросы вида `SELECT * FROM T1 JOIN T2 USING(Foo)`, где `Foo` - "расширяемое" поле

Способ организации вспомогательных полей, применяемый вышеописанным каркасом, "ломает" запросы со схемой как в заголовке данного подраздела. Проблема тут в том, что при выполнении миграции в таблицах `T1` и `T2` появятся одноименные поля `"_r64_Foo"` (т.к. в них уже было одноименное поле `Foo`, по которому производили объединение данных), которые, в свою очередь, запросом подобной структуры будут доставлены и на клиент. В результате клиентская часть не сможет построить по полученным данным объект типа `RecordSet` (т.к. все поля `RecordSet`-а должны иметь уникальные имена) и сгенерирует исключение. С самим полем `Foo`, по которому производится объединение, подобной проблемы не возникает, т.к. при использовании синтаксиса `USING` соответствующее поле оказывается в результирующем наборе данных в единственном экземпляре. Для решения проблемы подобные запросы придется переписать, явно перечислив список возвращаемых полей.

Невозможность создать вспомогательные объекты БД

Несмотря на то, что вышеописанный алгоритм обеспечивает минимальное влияние процесса миграции на текущую работу сервиса, ему на одном из этапов своей работы (путь и на очень короткий срок) требуется эксклюзивная (ACCESS EXCLUSIVE) блокировка на таблицу. Речь идет о самом начале работы, когда в таблице создаются вспомогательные поля и синхронизирующие триггеры (это делается в одной транзакции на колонку). Встречались ситуации, при которых в схемах клиентов или на отдельных БД велась столь интенсивная работа, что получить блокировку за адекватное время не удавалось. Время ожидания блокировки скриптом определяется параметром **"Ядро.КонвертерБД.ВремяОжиданияБлокировкиТаблицыСкриптомРасширенияПК"** (по умолчанию он равен 10000 мсек.), для каждой таблицы делается 100 попыток создания вспомогательных объектов. Итого, если за $100 * \text{"Ядро.КонвертерБД.ВремяОжиданияБлокировкиТаблицыСкриптомРасширенияПК"}$ объекты создать так и не удастся, скрипт выдает ошибку и прерывает свою работу. При возникновении подобной ошибки можно попробовать выставить в "Да" другой параметр конфигурации

- **"Ядро.КонвертерБД.ОтключениеКонкурирующихКлиентовСкриптомРасширенияПК"**. В этом случае скрипт будет пробовать отключить мешающих ему клиентов от БД при невозможности сразу наложить блокировку для исполнения DDL инструкций. Естественно, те запросы, которые исполняли отключенные клиенты, завершатся с ошибкой. При этом следует понимать, что т.к. отключение клиентов и последующая попытка получения блокировки не атомарны с т.з. СУБД, то и применение вышеописанного параметра не гарантирует успешности всей процедуры (хотя и сильно повышает шансы на успех). Если параметр не помогает, то решать проблему придется административными методами (договоренность с клиентом, блокировка клиента/сервиса и т.д.)

Блокировки, дедлоки и тормоза

Т.к. скрипт миграции в ходе своей работы массово обновляет записи в таблицах, возможны ситуации, при которых либо его инструкции блокируются какими-то внешними запросами, либо наоборот, его запросы мешают чьей-то работе. При обнаружении проблем подобного рода следует уменьшить агрессивность обновления записей (параметр BatchSize), либо параллелизм работы скрипта (параметр NumThreads). При этом следует понимать, что подобные действия приведут к увеличению (возможно, кратному) общего времени подготовки миграции. Если тюнинг настроек алгоритма не обеспечит нужного эффекта, то нужно будет провести более глубокий анализ возникающих блокировок и, возможно, внести какие-либо правки в прикладную кодовую базу.